



Rapport d'analyse : projet VeloScope

Équipe 14

Étudiants :

BONAVERO Méline
BOYCE Eddy
CHATONNIER Eliot
LEJARS Samuel
MOINERAUD-DOUCET Alix

Encadrant :

RIVIERE André

Table des matières

1	Introduction	1
2	Cahier des charges	1
2.1	Utilisateurs	1
2.2	Fonctionnalités de base attendues	1
2.3	Déroulé de la navigation	3
3	Modélisation	5
3.1	Architecture de l'API	5
3.1.a	Frontend	6
3.1.b	Backend	7
3.1.c	Couche persistente	7
3.2	Modèle de données	7
3.3	Classes	8
3.4	Description des endpoints	8
3.4.a	Comptes et authentification	9
3.4.b	Stations	9
3.4.c	Gestion des stations favorites	9
3.5	Diagramme de séquence d'une fonctionnalité	10
4	Organisation du projet	10
4.1	Calendrier prévisionnel	10
4.2	Outils de développement	10
	Annexes	11
	Bibliographie	12

1 Introduction

Le développement des mobilités douces est un axe majeur face au dérèglement climatique. Parmi les moyens de transports les plus utilisés actuellement figure le vélo. Cependant, l'accès à son propre vélo peut-être rendu difficile par différents facteurs socio-économiques (augmentation des prix face à une forte demande, amélioration technologique des vélos électriques lourd et cher, réduction des surfaces habitées par les ménages empêchant le stockage de son vélo personnel). C'est pourquoi toutes les villes se dotent de vélos en libre service, dont la gestion des stations est primordiale. Si de nombreuses applications existent déjà pour connaître l'état des stations de vélos en libre service, elles sont sujettes à de nombreux points faibles. Par exemple, elles ne permettent en effet de ne connaître que l'état actuel, et ne permettent pas de prévoir les disponibilités, même dans un futur proche. C'est pourquoi notre client souhaite développer une API qui irait bien plus loin. Elle permettrait d'étudier les stocks des différentes stations au cours du temps, et ainsi de connaître l'état des stations de manière fine. Elle permettrait aux usagers de pouvoir anticiper leur déplacement via des indicateurs d'évolution temporels et des recommandations adaptées.

2 Cahier des charges

L'objectif du projet est de produire une application donnant des informations sur la disponibilité des vélos dans une ville. Au-delà de la seule disponibilité en temps réel par station, l'application doit également fournir des indicateurs de qualité. Elle donnera notamment un indicateur de fiabilité, qui pourra potentiellement être visualisé grâce à une carte.

2.1 Utilisateurs

Deux types d'utilisateurs sont à distinguer. Il y aura tout d'abord les utilisateurs «réguliers» de l'application, qui devront créer un compte puis se connecter pour accéder à l'application. Une fois connectés, ils pourront accéder aux données de l'application et gérer leurs stations favorites. Ensuite, il y a aura des comptes «administrateurs» qui pourront gérer les comptes utilisateurs.

2.2 Fonctionnalités de base attendues

L'application possédera plusieurs fonctionnalités de base. Voici un diagramme de cas d'utilisation, que nous détaillons ci-dessous :



Figure 1: Diagramme de cas d'utilisation

Collecter des données via une API

La collecte des données concernant la disponibilité et l'état des stations de vélos se fera via une API externe. La collecte se fera à intervalle de temps régulier (toutes 15 min). Elle permettra de nourrir une base de données, hébergée ..., qui contiendra notamment le nom des stations, leurs localisations, leurs états et la disponibilité en terme de vélos et d'emplacement.

Consulter des données

Les utilisateurs pourront consulter, pour la station de leur choix, diverses informations. Premièrement, les données fournis par l'API (état de fonctionnement, disponibilité). Deuxièmement, des données calculées grâce à l'accumulation des données collectés (taux de remplissage, évolution de la disponibilité, durée moyenne des épisodes de saturation et de pénurie).

Fournir un score de fiabilité

Pour chaque station, l'application calculera et fournira un score de fiabilité, traduisant sa capacité à répondre de manière constante aux besoins des usagers. Ces derniers doivent donc pouvoir emprunter un vélo (station non vide) et le rendre (station non saturée). Pour évaluer cette fiabilité de façon pertinente, on s'appuie sur l'historique des états de la station sur une période donnée. Ce score intègre deux dimensions principales : la fréquence d'indisponibilité, mesurée par le pourcentage de temps passé à l'état vide ou saturé, et la durée moyenne des épisodes de pénurie ou de saturation. L'idée est donc de sanctionner la durée des pannes tout autant que leur fréquence, pour que l'application puisse mettre en avant les stations réellement fiables au quotidien.

Pour traduire cette logique mathématiquement, nous avons construit un score sur 100 points. Cette échelle présente l'avantage d'être intuitive pour l'utilisateur final lors de la recommandation de trajets. La formule sera établie par la suite, mais elle tiendra compte des variables suivantes : - les pourcentages de temps passé vides (Pvide) et saturés (Psat),

- les durées moyennes des épisodes de saturation (Msat) et de pénurie (Mvide).

La formule prévue sera de la forme : $\text{Score} = \max(0, 100 - f(\text{Pvide} ; \text{Psat} ; \text{Msat} ; \text{Mvide}))$

Recommander une station

L'application permettra de recommander des stations à proximité de l'utilisateur. Cette recommandation se basera à la fois sur la distance à l'utilisateur, sur l'historique de station (notamment via le score de fiabilité) et sur son historique récent.

2.3 Déroulé de la navigation

Voici ci-dessous un diagramme d'activité modélisant les différentes étapes de navigation dans l'appliation :

Diagramme d'Activité

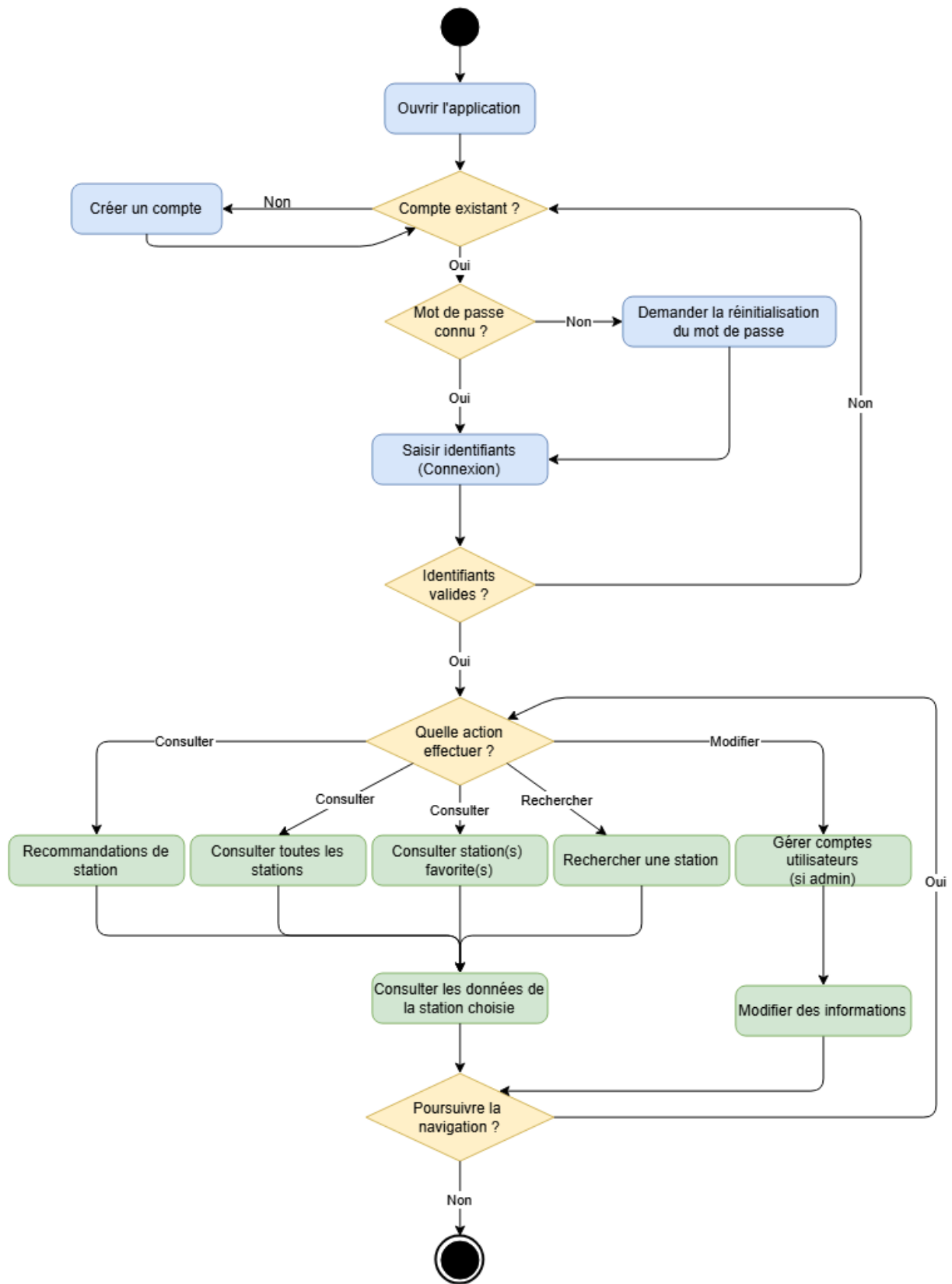


Figure 2: Diagramme d'activité

Gestion de compte

La première partie de ce diagramme correspond à la partie de gestion de compte de l'utilisateur. Elle lui permet de créer un compte si il en a pas, de modifier son mot de passe en cas d'oubli ou bien de se connecter à son compte. Seul l'administrateur dispose après connexion à son compte de la possibilité d'influer sur les comptes des utilisateurs "réguliers".

Fonctionnalité principales

L'ensemble des fonctionnalités de l'application sont dans la deuxième partie de ce diagramme. L'ensemble des options (recommandations, liste, favoris et recherches) mènent à la consultation des données liées à une seule station. L'utilisateur a ensuite le choix de quitter l'application ou de poursuivre la navigation en retournant à l'étape du choix d'action.

3 Modélisation

3.1 Architecture de l'API

L'API sera constituée de plusieurs couches afin de repsecter le principe de séparations des responsabilités. Voici un schéma d'ensemble de l'architecture logiciel envisagée, que nous détaillons dans la suite du dossier.

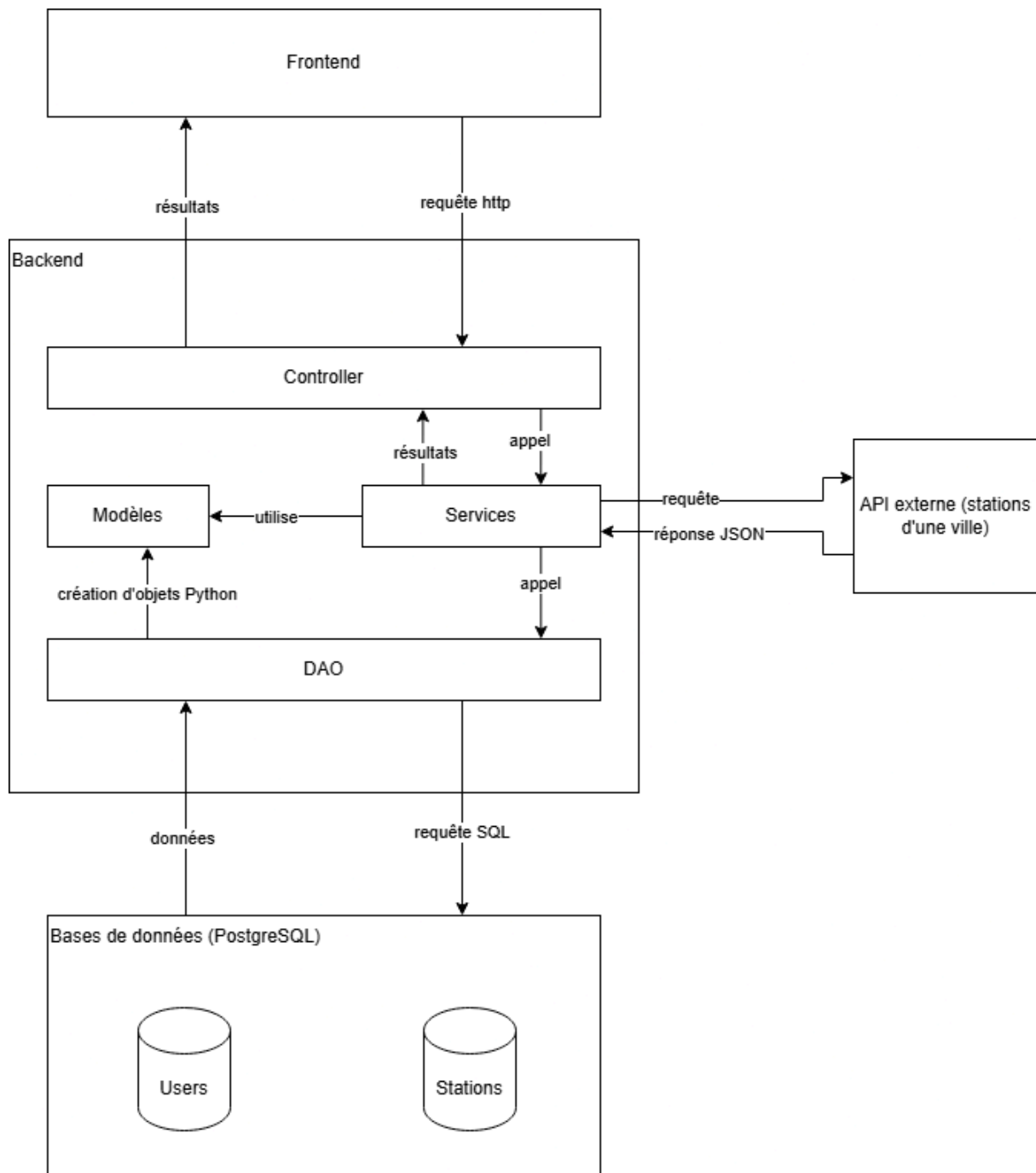


Figure 3: Diagramme d'architecture logiciel

3.1.a Frontend

Le *frontend* correspond à l'interface homme-machine. Il prendra la forme d'une application interactive accessible via une page Web.

L'interface se voudra simple et fonctionnelle : elle se présentera sous la forme d'une page HTML. Un menu à gauche permettra à l'utilisateur de sélectionner la fonctionnalité qu'il souhaite. Les différentes fonctionnalités disponibles porteront les noms suivants :

- rechercher une station,
- mes stations favorites,
- recommandation,

- gérer mon compte,
- gérer les comptes utilisateurs (n'apparaîtra que pour les administrateurs).

La gestion des stations favorites se fera de la manière suivante : l'utilisateur pourra ajouter de nouvelles stations lors de la consultation de la page d'une station. Il pourra ensuite consulter ses stations favorites ou les supprimer dans l'onglet "mes stations favorites".

3.1.b Backend

Le *frontend* enverra des requêtes http de la part du *backend*, qui sera chargé de traiter ces requêtes et de renvoyer la réponse attendue. Ces requêtes concerneront soit la gestion des utilisateurs (création de compte par exemple), soit la gestion des stations de vélos (consultation de la page d'une station par exemple). Le backend sera séparé en plusieurs couches.

Tout d'abord, une couche *controller* recevra les requêtes fournies par le frontend lors de l'utilisation de l'API par un usager. Cette couche aura pour but de convertir et transférer ces requêtes à la couche suivante, celles des services.

La couche *service* est le coeur fonctionnelle de l'API. C'est dans cette couche que seront regroupées les fonctions apportant une valeur ajoutée à notre API. Il s'agira par exemple des fonctions permettant de calculer les indicateurs souhaités et le score de fiabilité pour chaque station de vélos. Pour cela, la couche *service* a besoin de données. Ces services feront donc appel à une couche *DAO* (Data Access Object) afin de communiquer avec les bases de données.

La couche *DAO* recevra les demandes de données de la part de la couche *service*. Elle sera chargée de convertir ces demandes en requêtes SQL. Ces requêtes seront alors traitées par la couche persistente : celle des bases de données. La couche *DAO* aura alors pour mission de convertir en objets python (par exemple en objet *Utilisateur* ou *Station*, cf la section "Modèle de données" pour plus de détails). Ces objets seront ensuite utilisés par la couche *service* afin de calculer les indicateurs qui seront transmis à la couche *controller* afin de les afficher à l'utilisateur dans l'interface.

3.1.c Couche persistente

La couche persistente sera constituée des bases de données qu'utilisera notre API. Deux bases de données sont prévues : l'une pour les données utilisateurs, l'autre pour les données accumulées sur les stations de vélos.

3.2 Modèle de données

Afin que le modèle de données soit cohérent et pérenne, nous veillerons à respecter les trois premières formes normales.

Concernant les utilisateurs, une seule table "Users" sera nécessaire. Elle contiendra les données des utilisateurs : identifiant, pseudonyme de l'utilisateur, mot de passe hashé, statut (utilisateur simple ou administrateur).

Pour les stations, les données à stocker sont les suivantes : identifiant et nom de la station, coordonnées géographiques, état de la station, nombre d'emplacements, nombre d'emplacements libres, nombre de vélos disponibles (potentiellement électriques), date et heure. Le stockage des données au cours du temps nécessite d'être attentif aux formes normales. En effet, un stockage naïf des données nécessiterait une clé composite (id_station, date, heure). Cela créerait plusieurs problèmes :

- l'attribut nom_station serait en dépendance fonctionnelle avec id_station et donc avec seulement une partie de la clé primaire (contradiction avec le principe de la deuxième forme normale) ;
- l'attribut coordonnees_geo serait en dépendance fonctionnelle avec nom_station (contradiction avec

le principe de la troisième forme normale).

Il faudra donc deux tables. La première sera la relation **Stations**. Ses attributs stockeront les données concernant les stations elles-mêmes : l’identifiant de la station (clé primaire), son nom et ses coordonnées géographiques. La seconde sera la relation **StationRecords**, dont les attributs stockeront les données fournies par l’API externe. Voici les données stockées : l’identifiant de la station (attribut `id_station`), date et heure de l’enregistrement (attribut `date`), le nombre d’emplacements de la station (`nb_spaces`), le nombre d’emplacements libre de la station (`nb_available_spaces`), le nombre de vélos disponibles (`nb_available_bikes`) et le nombre de vélos électriques disponibles (`nb_available_ebikes`).

Enfin, l’un des attendus du projet est de permettre aux utilisateurs de pouvoir gérer leurs stations favorites. Cela se fera via une table d’association “Favorite” qui permettra de lier un utilisateur à ses stations favorites.

[Diagramme UML avec les données suivantes :]

3.3 Classes

D’après l’architecture logicielle, le projet nécessitera différents types de classes. Tout d’abord, il faudra implémenter les classes modèles “User”, “Station”, et “StationRecord” et “Favorite” qui modéliseront respectivement un utilisateur, une station, un enregistrement de station à une date précise et la liaison entre un utilisateur et ses stations favorites. Chacune de ces classes nécessitera une classe DAO propre afin de pouvoir faire la correspondance entre ces objets et les tables de données correspondantes. Il faut également implémenter des classes contrôller permettant de gérer les différents endpoints de l’API.

Chacune des quatre classes métiers prévues aura une classe “service” associée. Pour les classes “Users”, “StationRecord” et “Favorite”, les services permettront essentiellement de faire la jonction entre la couche “controller” et la couche “DAO” : ce seront donc des services purement techniques. La réelle plus-value de l’API se fera au niveau des services associés à la classe “Station”, notamment via le calcul d’indicateurs. Afin d’éviter d’avoir une classe avec trop nombreuses méthodes, et dans un but de factorisation du code, les services associés à la classe “Station” seront séparés en deux classes. Tout d’abord une classe “StationService” qui se concentre sur la logique métier, et qui permettra d’obtenir les indicateurs via une méthode `get_stats` qui fera appel à la classe `StationCalculationService`. Cette dernière sera chargée de calculer les indicateurs statistiques souhaités (taux de remplissage, disponibilité au cours du temps, pourcentage du temps passé vide ou saturé, temps moyen des épisodes de saturation et de pénurie, score de fiabilité).

La nécessité de l’utilisation de classe “schéma” utilisant “BaseModel” sera étudiée plus tard durant la phase de développement. Ces classes définiront notamment le format des Json échangés et permettront de faire le lien entre le backend et le frontend.

3.4 Description des endpoints

Les tableaux suivants décrivent les différents endpoints prévus de l’API.

3.4.a Comptes et authentication

Chemin	Méthode	Entrée	Sortie (succès)	Habilitation	Description
/create_user	POST	username, password, email	User	connecté	Crée un compte
/login	POST	username, password	jeton d'accès	connecté	Authentifie et renvoie un jeton
/users	GET	-	liste[User]	admin	Liste les comptes
/users/{id_user}	PATCH	username et/ ou password	User	admin	Modifie un compte
/users/{id_user}	DELETE	-	-	admin	Supprime un compte

3.4.b Stations

Chemin	Méthode	Entrée	Sortie (succès)	Habilitation	Description
/stations	GET	-	liste[Station]	connecté	Liste les stations
/stations/{id_station}	GET	-	Station	connecté	État actuel d'une station + indicateurs statistiques + score
/stations/{id_station}/history	GET	période (24h/7j/30j)	liste[Releve]	connecté	Évolution de la disponibilité
/suggestions	GET	position géographique	liste[Station]	connecté	Stations recommandées en fonction des critères

3.4.c Gestion des stations favorites

Chemin	Méthode	Entrée	Sortie (succès)	Habilitation	Description
/favorites	GET	-	liste[Station]	connecté	Favoris de l'utilisateur courant
/favorites/{station_id}	PUT	-	-	connecté	Ajoute une station aux favoris

Chemin	Méthode	Entrée	Sortie (succès)	Habilitation	Description
<code>/favorites/{station_id}</code>	DELETE	-	-	connecté	Retire une station des favoris

3.5 Diagramme de séquence d'une fonctionnalité

On présente ci-dessous le diagramme de séquence de la fonctionnalité “ajouter une station favorite”, afin de visualiser le lien entre les différentes couches. Ce diagramme traite du cas où la station n’est pas déjà en favori.

[Diagramme de séquence de la fonctionnalité “ajouter une station favorite”]

4 Organisation du projet

4.1 Calendrier prévisionnel

diagramme de Gantt

4.2 Outils de développement

Le projet sera réalisé en Python. Durant la phase de développement, nous utiliserons plusieurs outils et packages fournis pour ce langage. Voici le détail de ces outils pour chaque couche :

- le framework `streamlit` pour le *frontend* (afin de construire l’application Web interactive),
- le framework `FastAPI` pour le *backend*,
- le package `psycopg2` pour la couche *DAO* (afin de convertir les instructions Python en requêtes lisibles par les tables PostgreSQL).

De plus, afin de garantir une maintenance aisée, nous tenterons de respecter les bonnes pratiques de codage tant que faire se peut. Dans ce but, nous serons amenés à utiliser les outils suivants :

- Ruff pour le formatage du code,
- Pytest afin d’effectuer les tests unitaires,
- Github pour héberger et versionner notre projet.

Annexes

Bibliographie